

## 7. Variables & Their types

**NOTE:- GENERATED USING GPT**

### Variables & Scope

#### 1. Variables

- A variable is a **named memory location** used to store data.
- Each variable has:
  - **Name (identifier)**
  - **Type** (int, char, float, etc.)
  - **Value**

Example:

```
1  int x = 10;  
2  char c = 'A';
```

#### Key Points

- Type defines:
    - Memory size
    - Range of values
    - Allowed operations
  - C/C++ are **statically typed languages**
- 

#### 2. Scope

- Scope defines **the region of a program where a variable is accessible (visible)**.
  - It determines **where you can use a variable**.
- 

#### 3. Types of Scope

##### (a) Local Scope

- Declared inside a function or block `{}` .
- Accessible only within that block.

```
1 void func() {  
2     int x = 5; // local variable  
3 }
```

✓ Exists only inside the function/block

✗ Cannot be accessed outside

---

## (b) Global Scope

- Declared outside all functions.
- Accessible throughout the program.

```
1 int x = 10; // global variable  
2  
3 void func() {  
4     printf("%d", x);  
5 }
```

✓ Can be used anywhere

⚠ Can be modified anywhere → risky

---

## (c) Block Scope

- Declared inside `{}` like `if` , `for` , `while`

```
1 if (1) {  
2     int y = 20; // block scope  
3 }
```

✓ Only exists inside that block

---

## 4. Scope Rules

### Rule 1: Inner → Outer Access

- Inner scope can access variables from outer scope

```
1  int x = 10;
2
3  void func() {
4      printf("%d", x); // works
5  }
```

---

## Rule 2: Outer → Inner ❌

- Outer scope **cannot access inner variables**

```
1  if (1) {
2      int x = 5;
3  }
4  printf("%d", x); // error
```

---

## Rule 3: Shadowing

- Inner variable with same name overrides outer one

```
1  int x = 10;
2
3  void func() {
4      int x = 5; // shadows global x
5      printf("%d", x); // 5
6  }
```

---

## Rule 4: Function Scope Isolation

- Functions do NOT share local variables

```
1  int x = 10;
2
3  void func() {
4      printf("%d", x); // uses global
5  }
6
7  int main() {
8      int x = 20;
9      func(); // prints 10
```

## 5. Scope vs Lifetime

| Concept  | Meaning                      |
|----------|------------------------------|
| Scope    | Where variable is accessible |
| Lifetime | How long it exists in memory |

Example:

```
1 void func() {  
2     int x = 5;  
3 }
```

- Scope → inside `func`
- Lifetime → during function execution

## 6. Key Takeaways

- `{ }` defines most scopes
- Variables exist only inside their scope
- Inner scope can access outer variables (unless shadowed)
- Outer scope cannot access inner variables
- Functions have independent local scopes

## Mental Model

When accessing a variable, the compiler searches in this order:

1. Current block
2. Outer blocks
3. Global scope
4. ❌ Not found → error

## Scope Resolution Operator

# 1. Definition

- `::` is the **scope resolution operator**
  - It is used to **specify the exact scope** in which a variable or function exists
- 

## 2. Purpose

- Resolves **ambiguity caused by same names**
  - Helps access:
    - Global variables
    - Class members
    - Namespace members
- 

## 3. Accessing Global Variables

When a local variable shadows a global variable:

```
1  int x = 10;
2
3  int main() {
4      int x = 20;
5
6      cout << x << endl;    // 20 (local)
7      cout << ::x << endl; // 10 (global)
8  }
```

✓ `::x` → explicitly refers to **global x**

---

## 4. General Syntax

```
1  scope_name::identifier
```

Examples:

- `::x` → global variable
  - `ClassName::var` → class member
  - `namespace::var` → namespace member
-

## 5. With Classes

### Declaration inside class

```
1  class A {  
2  public:  
3      void show();  
4  };
```

### Definition outside class

```
1  void A::show() {  
2      cout << "Hello";  
3  }
```

✓ `A::show()` connects the function to class `A`

---

## 6. Without `::` (Important)

```
1  void show() {  
2      cout << "Hello";  
3  }
```

This becomes a **global function**, NOT part of class

---

## 7. With Namespaces

```
1  #include <iostream>  
2  
3  int main() {  
4      std::cout << "Hello";  
5  }
```

- `std` → namespace
  - `cout` → member inside namespace
- 

## 8. Key Use Cases

- Access global variables → `:: x`
  - Define class functions → `Class :: function`
  - Access namespace members → `std :: cout`
- 

## 9. Important Rules

- `::` removes ambiguity by specifying scope
  - It is required when defining class functions outside the class
  - It can access hidden (shadowed) global variables
- 

## 10. Scope vs `::`

- Without `::` → compiler searches automatically:
    1. Local scope
    2. Outer scope
    3. Global scope
  - With `::` → you **force the scope**
- 

## 11. Key Concept

`scope :: name` means  
“This name belongs to this specific scope”

---

## 12. Common Mistakes

- Forgetting `A::` when defining class functions → linker error
  - Confusing global and local variables when names are same
  - Assuming functions automatically belong to class
- 

## Final Takeaway

- `::` is used for:
  - **Access**

- Disambiguation
- Ownership (in classes)